

SHRI DADAJI INSTITUTE OF TECHNOLOGY AND TECHNOLOGY

Assignment - II (Answers)

Branch/Sem - CSE - IV Sem

Subject - Software Engineering (RGPV Syllabus)

Date of Assignment: 10/7/26

Date of Submission: 21/7/26

Q1. Describe fundamental Software design principles.

Software design translates the analysis model (requirements) into a design model that can be implemented. Good design is guided by a set of fundamental principles that help control complexity and improve quality, maintainability and reusability of software.

- **Abstraction:** Focusing on the essential characteristics of a component while suppressing unnecessary implementation detail. Data abstraction, procedural abstraction and control abstraction are common forms.
- **Modularity:** Dividing software into separate, independently developed and testable modules/components that together satisfy the requirements. Modularity reduces complexity and eases maintenance.
- **Architecture:** The overall structure of the software components, their properties, and the relationships/interactions among them.
- **Refinement (Stepwise Refinement):** A top-down design strategy in which a program is developed by successively refining levels of procedural detail, moving from a high-level statement of function to program statements.
- **Refactoring:** A reorganization technique that simplifies the design of a component without changing its function or external behaviour, improving internal structure.
- **Design Classes:** Identifying a set of classes (user-interface, business-domain, process, persistent, system classes) that together realize the software.
- **Information Hiding:** Modules should be designed so that information (data and procedure) contained within a module is inaccessible to other modules that have no need for such information — achieved through well-defined interfaces.
- **Functional Independence:** Achieved by designing modules with high *cohesion* (a module performs a single, well-defined task) and low *coupling* (minimal interdependence between modules). High cohesion and low coupling lead to software that is easier to develop, test and maintain.

Q2. Explain SA/SD approach to functional oriented design.

Structured Analysis/Structured Design (SA/SD) is a classical, function-oriented design approach that maps the data flow oriented analysis model, created using tools such as the Data Flow Diagram (DFD) and Data Dictionary, into a hierarchical program structure represented by a Structure Chart.

Structured Analysis (SA)

In this phase the system requirements are modelled as a set of DFDs that show how data flows through the system and is transformed by processes, along with data stores and external entities. A Data Dictionary describes every data item and store used in the DFDs.

Structured Design (SD)

The DFD is converted into a program (module) structure using two complementary mapping techniques:

- **Transform Mapping:** Used when the DFD exhibits a clear incoming flow, a central transform, and outgoing flow. The DFD is partitioned into these three parts and each part is mapped onto a corresponding branch of the structure chart, with the central transform modules called by a top-level controller module.
- **Transaction Mapping:** Used when the DFD is centred around a 'transaction centre' - a process from which several action paths (transactions) originate, depending on incoming data. Each transaction path is mapped to a separate module branch, controlled by a dispatcher module.

The resulting Structure Chart shows modules as boxes, calling relationships as arrows, and data/control flow between modules as labelled small arrows. The design is then evaluated and refined using measures such as coupling, cohesion, fan-in and fan-out.

Q3. How is system modelling achieved using UML? Explain with a suitable example.

The Unified Modeling Language (UML) is a standard, general-purpose modelling language used to visualize, specify, construct and document the artifacts of an object-oriented software system from different perspectives, using a set of diagram types.

Commonly used UML diagrams

- **Use Case Diagram:** Captures the functional requirements of the system by showing actors (users/external systems) and the use cases (services) the system provides them.
- **Class Diagram:** Shows the static structure of the system - classes, their attributes, operations, and relationships (association, aggregation, composition, inheritance).
- **Sequence Diagram:** An interaction diagram that shows how objects communicate over time by exchanging messages, emphasizing time ordering.
- **Activity Diagram:** Models the workflow/business process or the logic of a single operation as a flow of activities, decisions and parallel flows.
- **State Chart (State Machine) Diagram:** Models the different states an object can be in and the events/transitions that cause it to move from one state to another.
- **Component and Deployment Diagrams:** Show the physical organization of software components and their deployment on hardware nodes.

Example - Library Management System

In a Library Management System, a **Use Case Diagram** would show actors 'Student' and 'Librarian' connected to use cases such as 'Search Book', 'Issue Book' and 'Return Book'. A **Class Diagram** would define classes like Book, Member and Transaction with attributes (e.g., Book: title, ISBN, status) and associations (a Member borrows many Books through a Transaction). A **Sequence Diagram** for 'Issue Book' would show the Student object sending a request message to the Librarian interface, which in turn queries the Book object and updates the Transaction object - illustrating the exact order of message exchange needed to complete the use case.

Q4. Explain different kinds of architecture view and style.

Architectural Views (4+1 View Model)

Software architecture is often described using multiple, complementary views, each addressing a specific concern of different stakeholders. Kruchten's '4+1' model is widely used:

- **Logical View:** Describes the object model / key abstractions of the design (classes, objects) - relevant to end users and functionality.
- **Process View:** Captures the concurrency and synchronization aspects - the system's processes/threads and their interaction, addressing performance and scalability.
- **Development (Implementation) View:** Describes the software's static organization in the development environment - modules, layers, and subsystems - useful for programmers and software management.
- **Physical (Deployment) View:** Maps software components onto the physical hardware/network topology - relevant to system engineers.
- **Scenario View (+1):** A set of use cases/scenarios that illustrate and validate the architecture, tying the other four views together.

Architectural Styles

- **Data-Centered Architecture:** A central data store (e.g., a database or repository) is accessed and updated frequently by other components; suited to systems where data integration is central (e.g., ERP systems).
- **Data Flow Architecture:** Input data is transformed through a series of computational/manipulative components; includes the pipe-and-filter style, where each filter processes data and passes it via pipes to the next.
- **Call and Return Architecture:** Includes the main program/subprogram style (a top module calling subordinate modules) and layered architecture, where each layer provides services to the layer above it and uses services of the layer below (e.g., OSI model, OS design).
- **Object-Oriented Architecture:** Components encapsulate both data and the operations applied to that data, communicating with each other via message passing.
- **Event-Driven (Implicit Invocation) Architecture:** Components do not call one another directly; instead, a component broadcasts an event and other interested components register to receive and respond to it, common in GUI and distributed systems.

Q5. What is Black-Box testing? What is boundary value analysis?

Black-Box Testing

Black-box (behavioural) testing examines the functionality of an application without looking at its internal code structure. The tester treats the software as a 'black box' and derives test cases purely from the specification/requirements, checking that valid inputs produce correct outputs and invalid inputs are handled properly. It focuses on *what* the software does, not *how* it does it, and is typically performed at later stages such as system and acceptance testing. Common black-box techniques include equivalence partitioning, boundary value analysis, decision table testing, state-transition testing, and graph-based testing.

Boundary Value Analysis (BVA)

Boundary Value Analysis is a black-box test-case design technique based on the observation that errors tend to occur more often near the extreme edges (boundaries) of input domains rather than in the centre. Instead of choosing any element of an equivalence class, BVA selects test cases at, just above, and just below each boundary of the input/output ranges.

Example: If a field accepts an integer value in the range 1-100, BVA would generate test cases for the values 0, 1, 2 (just below, on, and just above the lower bound) and 99, 100, 101 (just below, on, and just above the upper bound), in addition to a typical mid-range value.

Q6. Explain different levels of software testing in detail.

- **Unit Testing:** Focuses on verifying the smallest testable unit of the software - a single module, class or function - in isolation. It checks internal logic, boundary conditions and error handling of the unit, typically performed by the developer using white-box techniques and stubs/drivers.
- **Integration Testing:** Performed after unit testing to verify that individually tested modules work correctly when combined. Its primary goal is to expose defects in the interfaces and interaction between integrated modules (using strategies like top-down, bottom-up, sandwich or big-bang integration).
- **System Testing:** Conducted on the complete, integrated system to evaluate its compliance with the specified requirements. It covers functional testing as well as non-functional aspects such as performance, security, load, recovery and usability testing.
- **Acceptance Testing:** The final level of testing, performed to determine whether the system satisfies the business/user requirements and is ready for delivery. It is usually carried out by (or on behalf of) the customer/end-user, and may take the form of Alpha testing (at the developer's site) or Beta testing (at the customer's site, in a real-world environment).

These levels form a testing hierarchy that progressively increases in scope, moving from a single unit to the fully integrated system operating in its intended environment, and together they help uncover different classes of defects at the earliest, cheapest point in the life cycle.

Q7. What are the Integration testing strategies?

Integration testing strategies define the order in which individual modules are combined and tested to build up the complete system.

- **Big-Bang Integration:** All (or most) modules are combined at once and tested as a complete system. It is simple but makes fault isolation difficult and is unsuitable for large systems.
- **Top-Down Integration:** Modules are integrated by moving downward through the control hierarchy, starting with the main/top module. Lower-level modules not yet integrated are simulated using *stubs*. This allows major control/decision logic to be tested early.
- **Bottom-Up Integration:** Integration begins with the lowest-level (terminal/leaf) modules, which are grouped into builds and tested using *driver* programs that simulate the calling higher-level modules not yet integrated. Major system control is tested last.
- **Sandwich (Hybrid) Integration:** Combines top-down and bottom-up strategies simultaneously - the system is viewed as three layers (target layer in the middle, tested using both stubs from above and drivers from below), gaining the advantages of both approaches.
- **Regression Testing:** Not a separate integration order but a supporting activity performed after each new module or change is integrated, re-running existing test cases to ensure that new integration has not introduced defects into previously working functionality.

Q8. Explain static and dynamic Analysis with the importance of code inspection.

Static Analysis

Static analysis examines software artifacts - source code, design documents or requirements - without actually executing the program. It relies on manual review techniques (walkthroughs, inspections) and/or automated static-analysis tools that check for coding-standard violations, unreachable code, uninitialized variables, type mismatches and other structural defects. Because it does not require a running system, static analysis can be applied very early in development and is

effective at finding defects cheaply, before they propagate to later phases.

Dynamic Analysis

Dynamic analysis evaluates software by executing it on a set of test data and observing its actual behaviour, output and performance (this is essentially what testing does). It can uncover run-time defects such as memory leaks, incorrect computations, race conditions and performance bottlenecks that static analysis cannot reveal, since these depend on the program's actual execution path and environment.

Importance of Code Inspection

Code inspection (e.g., the formal Fagan Inspection process) is a key static analysis technique in which a small team of reviewers systematically examines source code against a checklist of common error types, with defined roles (moderator, author, reader, recorder).

- Detects defects very early, when the cost of fixing them is lowest, well before execution/testing.
- Finds classes of errors (e.g., logic errors, standards violations, security flaws) that may be missed by dynamic testing because they do not manifest for the particular test data used.
- Improves code quality, readability and adherence to coding standards through peer review.
- Serves as a knowledge-sharing mechanism, spreading understanding of the code among team members and mentoring less experienced developers.
- Provides objective, measurable data (defect density, inspection rate) that can be used to improve the overall software process.